

DEVSECOPS TOOLS INSTALL & USAGE GUIDE

Setup commands and basic usage for the industry-standard DevSecOps toolchain

PLAN

Jira

Issue tracking & agile project management for sprint planning.

INSTALL

Sign up at <https://www.atlassian.com/software/jira>
No local install – SaaS or self-hosted Data Center option

USAGE

Create Project > Board > Sprint
jira-cli: `jira issue create -t Task -s "Fix XSS"`

OWASP Threat Dragon

Free, open-source threat modeling tool (STRIDE).

INSTALL

```
docker pull owasp/threat-dragon
docker run -p 3000:3000 owasp/threat-dragon
```

USAGE

Open <http://localhost:3000>
Create model > Add diagram > Identify threats per STRIDE category

CODE

Git

Distributed version control system — foundation of the pipeline.

INSTALL

```
sudo apt install git -y # Debian/Ubuntu
brew install git # macOS
```

USAGE

```
git init && git add . && git commit -m "init"
git remote add origin && git push -u origin main
```

GitLeaks

Scans git repos/commits for hardcoded secrets and credentials.

INSTALL

```
brew install gitleaks
# or: docker pull zricethezav/gitleaks
```

USAGE

```
gitleaks detect --source . -v
gitleaks protect --staged # pre-commit check
```

TruffleHog

Deep secret scanning across git history and cloud storage.

INSTALL

```
pip install truffleHog3 --break-system-packages
# or: docker pull trufflesecurity/trufflehog
```

USAGE

```
trufflehog git file:/// --only-verified
trufflehog github --repo=https://github.com/org/repo
```

pre-commit

Framework to run linters/scanners automatically before each commit.

INSTALL

```
pip install pre-commit --break-system-packages
```

USAGE

```
pre-commit sample-config > .pre-commit-config.yaml
pre-commit install && pre-commit run --all-files
```

BUILD

Jenkins

Open-source automation server for CI/CD pipelines.

INSTALL

```
docker run -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

USAGE

```
Open http://localhost:8080, unlock with initial admin password
Create Pipeline job > add Jenkinsfile with stages{} block
```

SonarQube

SAST platform for code quality & security hotspots.

INSTALL

```
docker run -d -p 9000:9000 sonarqube:community
```

USAGE

```
Login at http://localhost:9000 (admin/admin)
sonar-scanner -Dsonar.projectKey=myapp -Dsonar.sources=.
```

Semgrep

Fast, lightweight static analysis with custom rule support.

INSTALL

```
pip install semgrep --break-system-packages
# or: brew install semgrep
```

USAGE

```
semgrep --config=auto .
semgrep --config=p/owasp-top-ten .
```

Snyk

Dependency (SCA), container, and IaC vulnerability scanning.

INSTALL

```
npm install -g snyk
snyk auth
```

USAGE

```
snyk test # scan dependencies
snyk container test myimage:latest
snyk iac test main.tf
```

OWASP Dependency-Check

Identifies known-vulnerable dependencies (CVE match).

INSTALL

```
brew install dependency-check
# or download release zip from OWASP GitHub
```

USAGE

```
dependency-check.sh --project myapp --scan ./ --format HTML
```

TEST

OWASP ZAP

Free DAST proxy/scanner for web application security testing.

INSTALL

```
docker pull zaproxy/zap-stable
```

USAGE

```
docker run -t zaproxy/zap-stable zap-baseline.py -t https://example.com
zap-full-scan.py -t https://example.com -r report.html
```

Burp Suite

Industry-standard manual + automated web app pentest proxy.

INSTALL

```
Download from https://portswigger.net/burp/communitydownload
java -jar burpsuite_community.jar
```

USAGE

```
Set browser proxy to 127.0.0.1:8080
Use Proxy > Intercept and Scanner (Pro) to test requests
```

Postman

API testing, incl. auth, fuzz & security test collections.

INSTALL

```
Download from https://www.postman.com/downloads
```

USAGE

```
Import OpenAPI spec > Build collection
Add pre-request/test scripts for auth & security assertions
```

RELEASE / PACKAGE

JFrog Artifactory

Universal artifact repository manager (Docker, npm, Maven...).

INSTALL

```
docker run -d -p 8081:8081 releases-docker.jfrog.io/jfrog/artifactory-oss:latest
```

USAGE

Configure repo at <http://localhost:8082>

```
mvn deploy -DaltDeploymentRepository=artifactory::default::http://localhost:8081/...
```

Trivy

All-in-one scanner: containers, IaC, filesystems, SBOM.

INSTALL

```
brew install aquasecurity/trivy/trivy  
# or: sudo apt install trivy
```

USAGE

```
trivy image myapp:latest  
trivy fs .  
trivy config ./terraform
```

Cosign (Sigstore)

Signs and verifies container images & software artifacts.

INSTALL

```
brew install cosign  
# or: go install github.com/sigstore/cosign/v2/cmd/cosign@latest
```

USAGE

```
cosign generate-key-pair  
cosign sign --key cosign.key myrepo/myapp:latest  
cosign verify --key cosign.pub myrepo/myapp:latest
```

DEPLOY

Terraform

Infrastructure as Code — provisions cloud resources declaratively.

INSTALL

```
brew install terraform  
# or: sudo apt install terraform
```

USAGE

```
terraform init  
terraform plan -out=tfplan  
terraform apply tfplan
```

Checkov

Static analysis for IaC (Terraform, CloudFormation, K8s, ARM).

INSTALL

```
pip install checkov --break-system-packages
```

USAGE

```
checkov -d ./terraform  
checkov -f main.tf --compact
```

Ansible

Agentless configuration management & deployment automation.

INSTALL

```
pip install ansible --break-system-packages
```

USAGE

```
ansible-playbook -i inventory.ini site.yml
ansible all -m ping -i inventory.ini
```

Kubernetes (kubectl) + Helm

Container orchestration and package management.

INSTALL

```
curl -LO "https://dl.k8s.io/release/stable.txt" # kubectl
brew install helm
```

USAGE

```
kubectl apply -f deployment.yaml
helm install myapp ./chart -f values.yaml
```

OPERATE & SECURE INFRA

HashiCorp Vault

Centralized secrets management, dynamic secrets, encryption.

INSTALL

```
brew install vault
# or: docker run -p 8200:8200 hashicorp/vault
```

USAGE

```
vault server -dev # dev mode
vault kv put secret/myapp password=s3cr3t
vault kv get secret/myapp
```

Open Policy Agent (OPA)

Policy-as-code engine — enforce rules across the stack.

INSTALL

```
brew install opa
# or: curl -L -o opa https://openpolicyagent.org/downloads/latest/opa_linux_amd64
```

USAGE

```
opa eval -i input.json -d policy.rego "data.example.allow"
opa test policy_test.rego
```

Falco

Runtime security — detects anomalous behavior in containers/K8s.

INSTALL

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm install falco falcosecurity/falco
```

USAGE

```
kubectl logs -l app=falco -n falco -f
Customize rules in falco_rules.local.yaml
```

Kube-bench

CIS benchmark compliance checks for Kubernetes clusters.

INSTALL

```
docker pull aquasec/kube-bench:latest
```

USAGE

```
kube-bench run --targets master,node
kube-bench run --benchmark cis-1.24
```

MONITOR

Prometheus + Grafana

Metrics collection and visualization/alerting dashboards.

INSTALL

```
docker run -p 9090:9090 prom/prometheus
docker run -p 3000:3000 grafana/grafana
```

USAGE

```
Open http://localhost:9090 to query metrics (PromQL)
Open http://localhost:3000, add Prometheus as data source, build dashboards
```

ELK Stack

Elasticsearch + Logstash + Kibana for centralized log analytics.

INSTALL

```
docker-compose -f elk-docker-compose.yml up -d
```

USAGE

```
Ship logs via Filebeat/Logstash pipeline
Explore & visualize in Kibana at http://localhost:5601
```

Wazuh

Open-source SIEM/XDR — host intrusion detection & compliance.

INSTALL

```
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

USAGE

```
Access dashboard at https://
Deploy wazuh-agent on endpoints for monitoring
```

PagerDuty

Incident alerting, on-call scheduling & escalation (SaaS).

INSTALL

```
Sign up at https://www.pagerduty.com — no local install
```

USAGE

```
Create a Service > Integration (e.g. Prometheus/Grafana webhook)
Configure escalation policy for alert routing
```